人形机器人综合实践

第二讲

ROS机器人操作系统

学堂在线智能机器人
课程课件，仅供学习，

课程概要

- ROS 简介
  - 什么是 ROS？为什么使用 ROS，ROS1 or ROS2？
- ROS 通信机制
  - 话题，服务
- ROS 开发基础
  - 工作空间，功能包，roslaunch …
- ROS 常用工具
  - RQT， TF， RVIZ…
- 项目实践

学堂在线智能机器人创新人才培养计划
课程课件，仅供学习，严禁外传

Contents

- **ROS 简介**
- ROS 通信机制
- ROS 开发基础
- ROS 常用工具
- 项目实践

学堂在线智能机器人创新人才培养计划
课程课件，仅供学习，严禁外传

ROS 是什么？操作系统？

ROS (Robot Operating System) provides libraries and tools to help software developers create robot applications. It provides hardware abstraction, device drivers, libraries, visualizers, message-passing, package management, and more.

——ROS Wiki

- **ROS 是一个中间件：**

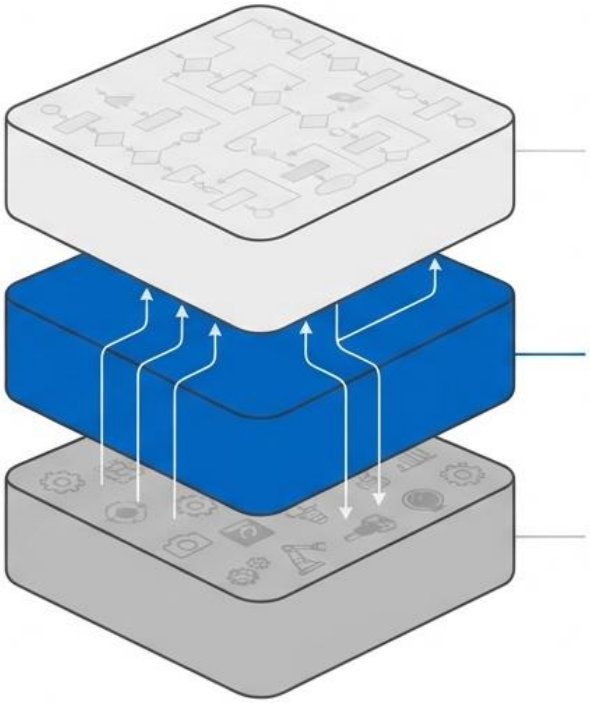
  连接硬件与上层应用的“管道”

  提供了一套通信框架、工具链和生态库

- **核心理念**

  避免重复造轮子

- **类操作系统功能**



创新人才培养计划 严禁外传 学堂在线智能 课程课件

ROS 核心架构：分布式节点网络

- **节点（Node）：**

  一个执行具体任务的进程

  如驱动激光雷达、规划路径、控制电机

- **管理器（Master）**

  通过 roscore 启动

  负责节点的注册与发现，类似于电话簿或DNS服务器

- **分布式（Distributed）**

  节点可在不同计算机上运行，通过网络通信

学堂在线智能机器人创新人才培养计划
课程课件，仅供学习，严禁外传

ROS1 vs ROS2

	ROS1 (Classic)	ROS2 (Future)
架构	中心化，依赖 Master	去中心化，基于 DDS
通信	TCP/IP	DDS（支持多种实现）
实时性	较弱	支持实时系统
适用场景	科研、原型开发	工业部署、产品化
启动方式	需要先运行 roscore	直接启动节点

本课程以 ROS1 为例讲解核心概念
ROS2 的节点/话题/服务等概念基本一致，技术迁移简易

Contents

- ROS 简介
- **ROS 通信机制**
- ROS 开发基础
- ROS 常用工具
- 项目实践

学堂在线智能机器人创新人才培养计划
课程课件，仅供学习，严禁外传

ROS 三大通信模型概览

机器人如何交流？

话题（Topic）

- **模型**：异步广播（一对多）
- **场景**：连续数据流，如相机图像、雷达数据

(将首先聚焦于此)

服务（Service）

- **模型**：同步请求/响应（一对一）
- **场景**：触发一次性动作，如查询状态

动作（Action）

- **模型**：带反馈的长时任务
- **场景**：需要过程监控的任务，如导航到目标点

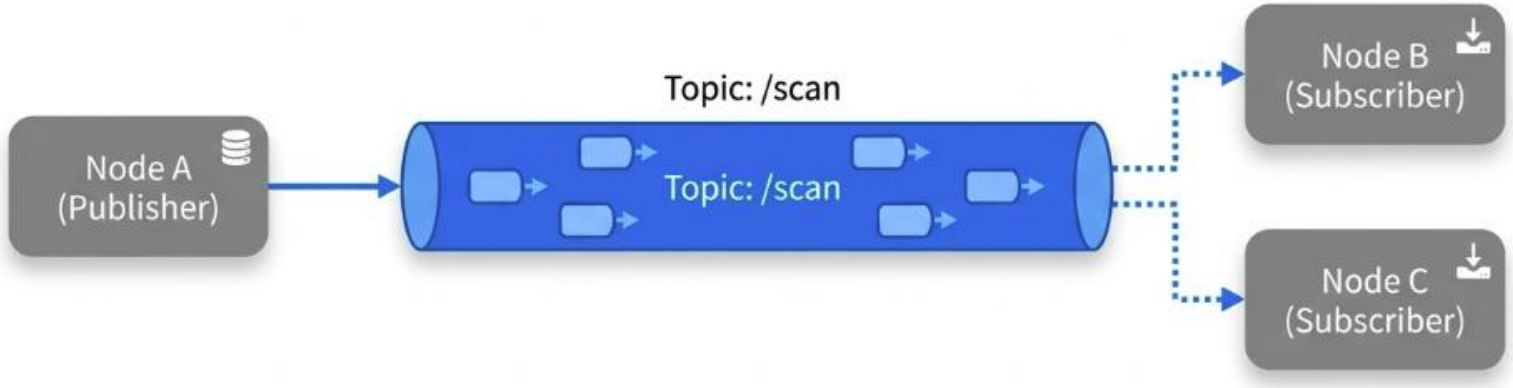
智能机器人创新人才培养计划 仅供学习，严禁外传

Topic 机制解析

单向广播的数据流管道

- 节点（发布者 Publisher）将数据打包成消息（Message），发布到指定话题（Topic）上
- 任意数量的节点（订阅者 Subscriber）可以订阅该话题，以接收消息
- 发布者和订阅者之间完全解耦，只关心话题，互不感知对方的存在

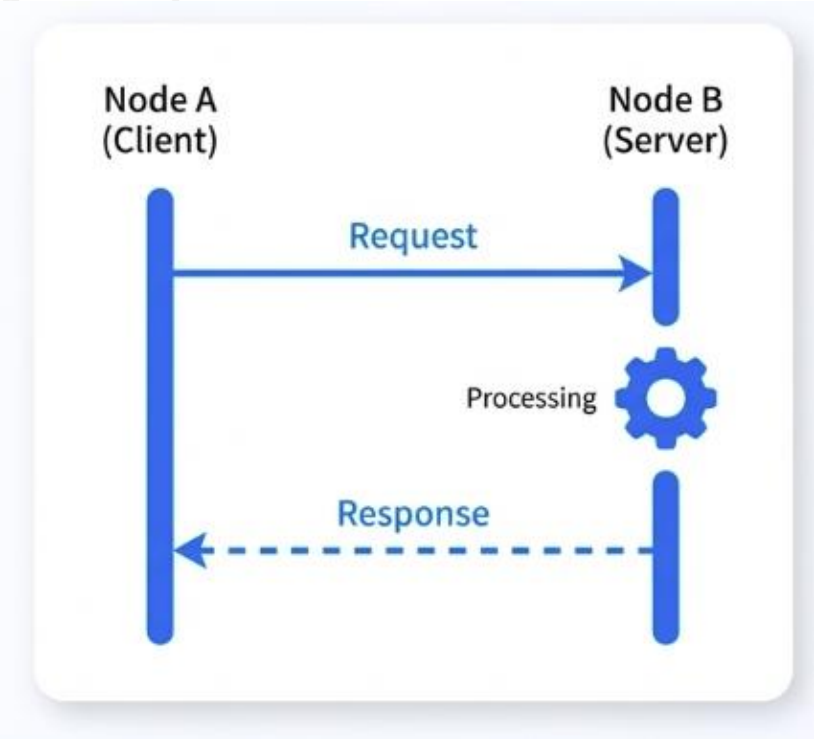
学生在线看华为机器人创新人才培养计划课程课件，仅供学习，严禁外传



Service 机制解析

一问一答式的阻塞式请求

- **Client（客户端）**：发送一个请求（Request）并等待答复
- **Server（服务端）**：接收请求，处理任务，然后返回一个反馈（Response）
- **阻塞式特性（Blocking）**：客户端必须等待服务端完成任务并返回结果，才能继续执行下一步



学堂在线智能机器人创新人才培养计划
仅供学习课程课件，严禁外传

动手实验1：启动你的首个机器人

ROS 通用启动流程

1. 启动 ROS 核心（新终端 1）

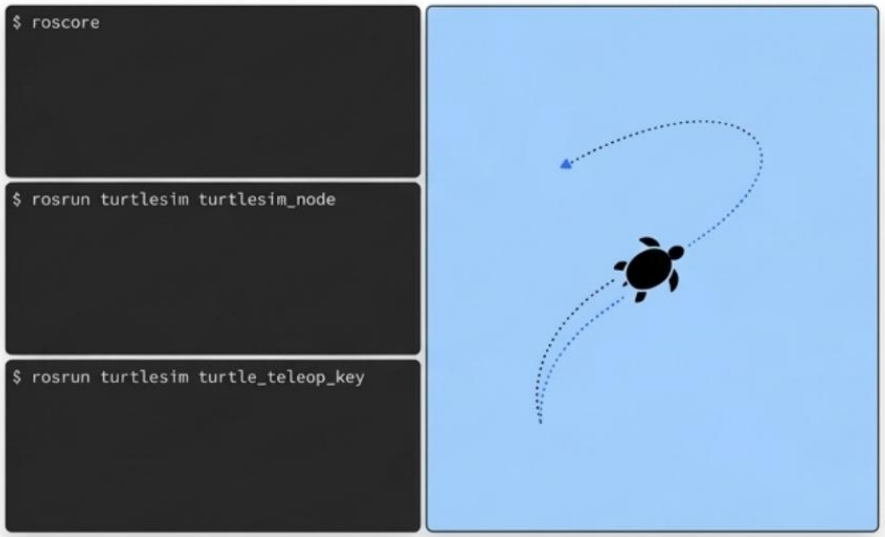
```
$ roscore
```

2. 启动小海龟仿真器（新终端 2）

```
$ rosrun turtlesim turtlesim_node
```

3. 启动键盘控制器（新终端 3）

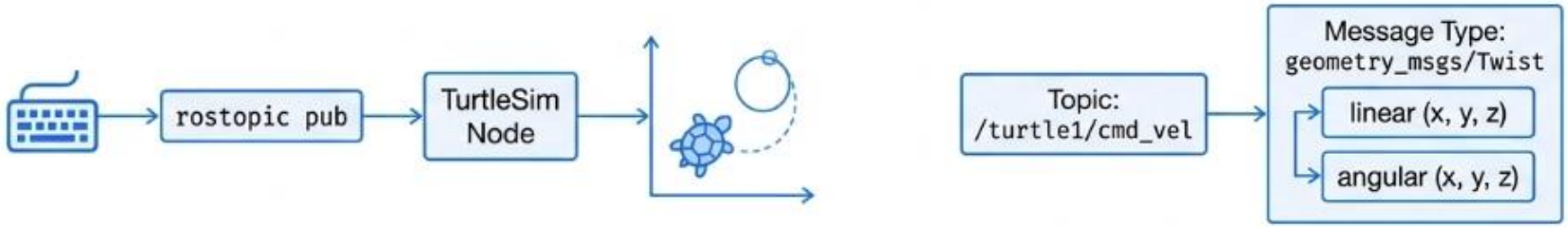
```
$ rosrun turtlesim turtle_teleop_key
```



Action：在键盘控制终端，使用方向键控制小海龟移动

实验效果解析

- **通信结构梳理**
  - turtle_teleop_key 是一个**发布者**
  - turtlesim_node 是一个**订阅者**
  - 它们通过 /turtle1/cmd_vel 这个**话题**进行通信
  - **消息**类型为 geometry_msgs/Twist，定义了线速度和角速度
- **利用 ROS 工具查看**
  - 查看节点关系图（新终端）：rqt_graph （查看节点连接关系）
  - 查看消息结构：rosmsg show geometry_msgs/Twist
- 尝试使用命令行 rostopic pub 手动发布速度指令



学生在线智能机器人创新人才培养计划 课程课件，仅供学习，严禁外传

Contents

- ROS 简介
- ROS 通信机制
- **ROS 开发基础**
- ROS 常用工具
- 项目实践

学堂在线智能机器人创新人才培养计划
课程课件，仅供学习，严禁外传

工作空间

从临时命令到专业项目：建立工作空间(Workspace)

What it is: 一个专门用于存放、编译和管理你所有 ROS 功能包的目录结构。

标准结构（The Catkin Way）：

src/：源码区（你的代码放这里）

build/：编译中间文件（自动生成）

devel/：开发空间（包含目标文件和环境脚本）

```
➜ catkin_ws$ tree
catkin_ws/
├── build/        // 编译中间文件（自动生成）
├── devel/        // 开发空间（目标文件和环境脚本）
└── src/          // 源码区（你的代码放这里）

➜
```

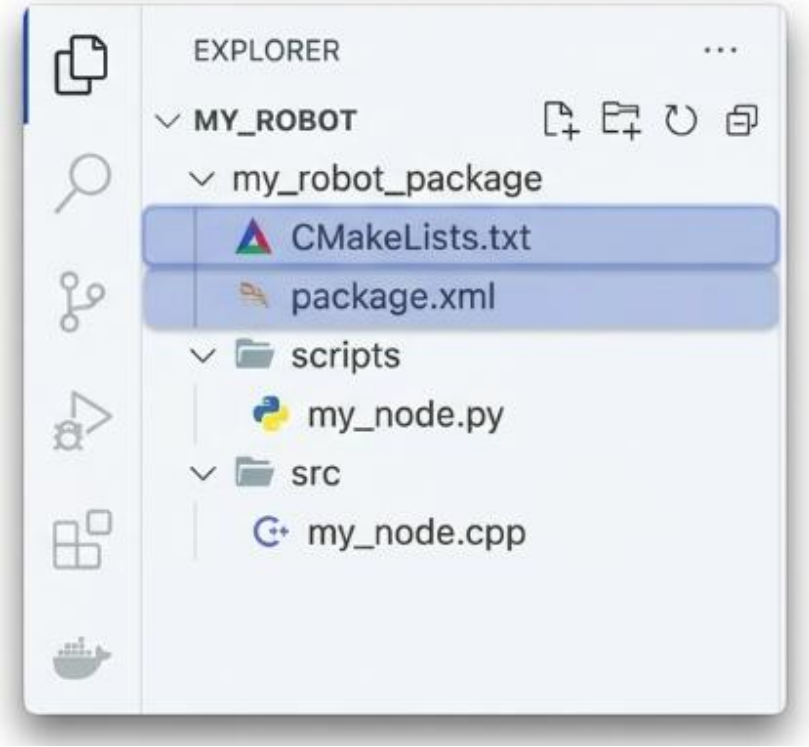
The Professional Workflow:

1. **创建目录：** `mkdir -p ~/catkin_ws/src`
2. **进入并编译：** `cd ~/catkin_ws && catkin_make`
3. **激活环境：** `source devel/setup.bash`（关键步骤！它告诉终端去哪里找你的新程序）

创新人才培养计划 严禁外传

ROS 功能包

- **Package.xml**：声明功能包的名称、作者、依赖项等元信息
- **CMakeLists.txt**：指导编译器如何编译代码、链接库、生成可执行文件
- **scripts/ & src/**：分别用于存放 Python 和 C++ 源码



光学在线智能机器人创新人才培养计划
课程课件，仅供学习 严禁外传

roslaunch 一键启动ros系统

- **批量启动**：无需手动打开几十个终端，一个命令启动多个节点（含roscore）
- **参数管理**：自动加载配置参数到ROS 参数服务器
- **灵活配置**：可以轻松重命名节点、设置命名空间等参数

```
<launch>
  <!-- Start the camera driver node -->
  <node pkg="my_camera_pkg" type
="camera_node" name="camera_driver" />

  <!-- Load calibration parameters -->
  <param name="camera_calibration_file"
value="$(find my_pkg)/config/calib.
yaml" />
</launch>
```

学堂在线智能机器人创新人才培养计划 仅供学习，严禁外传

Contents

- ROS 简介
- ROS 通信机制
- ROS 开发基础
- **ROS 常用工具**
- 项目实践

学堂在线智能机器人创新人才培养计划
课程课件，仅供学习，严禁外传

可视化工具箱

RQT Graph (rqt_graph)

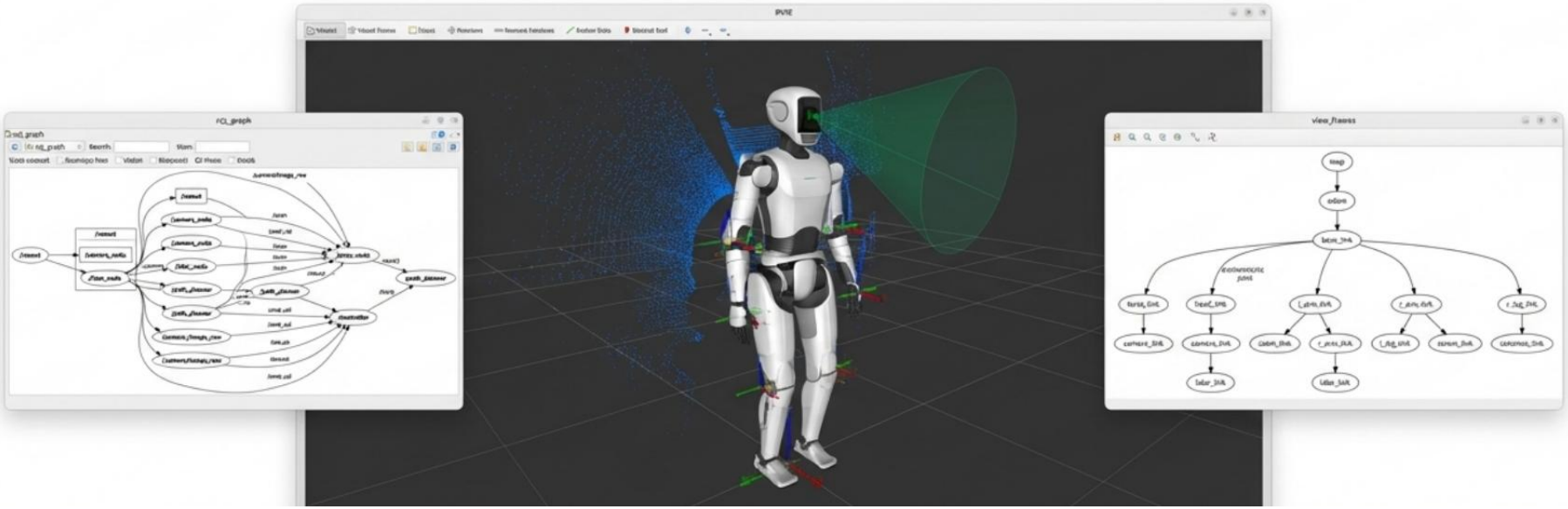
- **功能**：可视化节点关系图
- **用途**：查看节点连接关系是否正确，探索 rqt 更多用法

TF (view_frames)

- **功能**：显示坐标变换关系
- **用途**：查看机械臂末端相对于基座的位姿

RVIZ (rviz)

- **功能**：3D可视化工具
- **用途**：直观显示机器人模型、传感器数据、规划路径等



机器人创新人才培养计划 严禁外传

Contents

- ROS 简介
- ROS 通信机制
- ROS 开发基础
- ROS 常用工具
- **项目实践**

学堂在线智能机器人创新人才培养计划
课程课件，仅供学习，严禁外传

动手实验2：话题通信机制

创新人才培养计划 严禁外传

构建一个图像传输系统

Mission: 创建两个节点，一个发布本地图片，另一个接收并显示。

Step 1: 创建功能包

- 进入 `src` 目录：`cd ~/catkin_ws/src`
- 使用 `catkin_create_pkg` 创建一个名为 `image_tutorial` 的包，并声明其依赖：

```
$ catkin_create_pkg image_tutorial rospy std_msgs sensor_msgs
```

Step 2: 添加并准备代码

```
catkin_ws/
└ src/
  └ image_tutorial/
    ├ src/
      ├ pub_image.py ←
      └ sub_image.py ←
    ├ CMakeLists.txt
    └ package.xml
```

- 将我们提供的 `pub_image.py` 和 `sub_image.py` 放入 `~/catkin_ws/src/image_tutorial/src/` 目录。
- **重要：** 修改 `pub_image.py` 中的图片路径为你电脑上的**绝对路径**。
- 赋予脚本执行权限：

```
$ cd ~/catkin_ws/src/image_tutorial/src/
$ chmod +x *.py
```

动手实验2：话题通信机制

创新人才培养计划 严禁外传

编译并运行视觉传输系统

Step 3: 编译工作空间

回到工作空间根目录并编译：

```
$ cd ~/catkin_ws
$ catkin_make
```

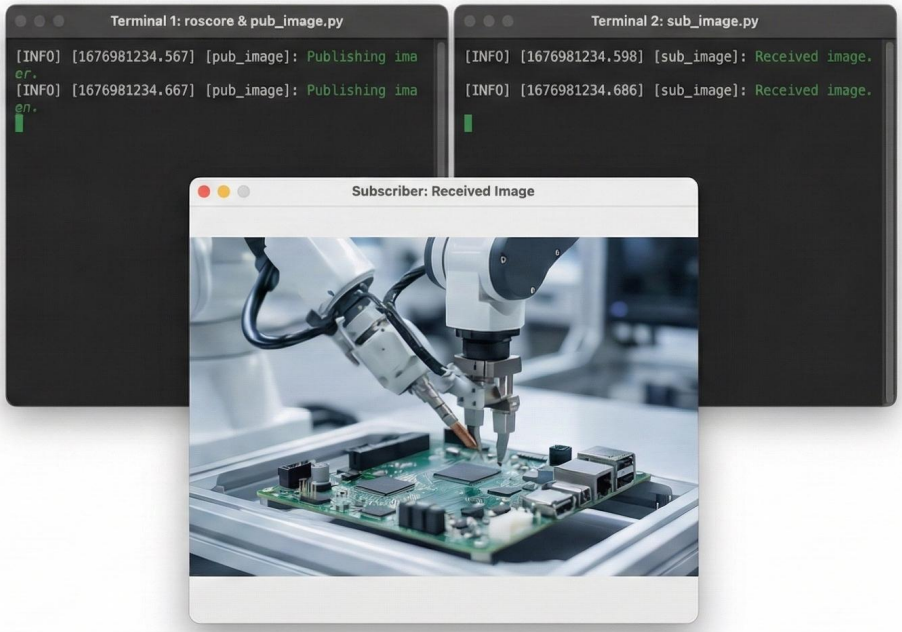
Step 4: 运行节点

重要：每次编译后或新开终端，都要激活环境！

```
$ source devel/setup.bash
```

- **启动核心（终端 1）：**`roscore`
- **运行发布者（终端 2）：**`rosrun image_tutorial pub_image.py`
- **运行订阅者（终端 3）：**`rosrun image_tutorial sub_image.py`

Expected Result:



订阅者终端会弹出一个窗口，显示发布者正在发送的图片。

常见问题速查

视觉传输系统解析:

- pub_image.py: 一个读取图片并作为 sensor_msgs/Image 消息发布的**发布者**
- sub_image.py: 一个订阅图像话题，并将消息转换回图片显示的**订阅者**

常见 bug 速查手册

问题：`[rosrun] Couldn't find executable...`

解决方案：忘记 `source ~/catkin_ws/devel/setup.bash` 了吗？`catkin_make` 成功了吗？包名和节点名拼写正确吗？

问题：脚本报错 `No such file or directory...`

解决方案：确认 `pub_image.py` 中的图片路径是绝对路径并且文件真实存在。

问题：编译后代码不生效

解决方案：尝试清理后重编译。删除`build`和`devel`目录，然后重新运行 `catkin_make`。

学堂在线智能机器人创新人才培养计划 课程课件，仅供学习，严禁外传